Technische Universität Berlin

Chair of Database Systems and Information Management

Master's Thesis

Hardware-conscious Performance Engineering for Distribution-Aware Dataset Search

A Re-examination of Fainder for hardware-near optimization

Tarik Abu Mukh, t.abu.mukh@campus.tu-berlin.de
Degree: M.Sc. Computer Science
Matriculation Number: 0501953

Reviewers
Prof. Dr. Volker Markl
Prof. Dr. Rieck

Advisor(s)
Lennart Behme

Submission Date
July 15, 2026

Declaration of Authorship

I hereby declare that the thesis submitted is my own, unaided work, completed without any external help. Only the sources and resources listed were used. All passages taken from the sources and aids used, either unchanged or paraphrased, have been marked as such. Where generative AI tools were used, I have indicated the product name, manufacturer, the software version used, as well as the respective purpose (e.g. checking and improving language in the texts, systematic research). I am fully responsible for the selection, adoption, and all results of the AI-generated output I use.

I have taken note of the Principles for Ensuring Good Research Practice at TU Berlin dated 15 February 2023.

I further declare that I have not submitted the thesis in the same or similar form to any other examination authority.

Berlin, July 15, 2026

.....

Tarik Abu Mukh

Documentation of Tool Usage

In accordance with the requirements for good scientific practice, below I have documented my use of (AI) tools in the process of writing this thesis.

Phase	**Level of Use**	**Used Tool(s)**	**Link to Prompts**
Finding and nailing down the Topic			
Understanding the topic			
Literature Research			
Implementing the Prototype	Moderate	Claude Code, GitHub Copilot	—
Testing the Prototype	Moderate	Claude Code	—
Planning the Thesis			
Writing The thesis			
Revising the Thesis			

Scope of AI tool usage. Claude Code (Anthropic) and GitHub Copilot were used during the implementation and testing phases. Specifically, the tools were used for:

- generation of matplotlib plotting code for the experimental figures reproduced in the walkthrough notebooks,
- analysis of the SQLite measurement database (`logs/bench.db`), including SQL query generation for cell-by-cell inspection of the ablation matrix,
- construction of the perf-counter logging and post-processing scripts that populate the measurement database from raw `perf stat` output,
- routine boilerplate scaffolding (Cargo feature-gating, PyO3 binding stubs, shell script structure).

The scientific claims, the four-ceiling characterisation, the negative-composability pattern, the pre-flight ceiling-identification methodology, the dispatch policy, and their evaluation are the author's own work. All ablation choices, hypothesis formulations, mechanism explanations, and interpretations of the empirical results were formulated by the author independently of AI tools.

Zusammenfassung

Verteilungsbasierte Datensatzsuche adressiert die Aufgabe, Datensätze anhand ihrer statistischen Eigenschaften zu finden, wenn die Rohdaten nicht zugänglich sind und lediglich Metadaten in Form von spaltenweisen Histogrammen vorliegen. Fainder [2] ist ein kürzlich vorgeschlagener Index für dieses Problem: er clustert Histogramme mit K-Means, richtet sie innerhalb jedes Clusters auf ein gemeinsames Bin-Gitter aus und berechnet kumulative Dichten vor, sodass Perzentilprädikate durch Binärsuche anstelle eines vollständigen Histogrammdurchlaufs beantwortet werden. Auf der GitTables-Kollektion mit fünf Millionen Histogrammen erreicht Fainder eine mehr als zweihundertfache Beschleunigung gegenüber der `profile-scan`-Baseline und reduziert den Speicherbedarf von über 1 TB auf 3,2 GB. Die Referenzimplementierung ist jedoch in Python geschrieben, und der Python-Overhead dominiert ihre Abfragezeit: der Global Interpreter Lock serialisiert die Cluster-Schleife über Threads hinweg, das Array-of-Structs-Layout von NumPy verschmutzt Cache-Lines mit ungelesenen Histogramm-ID-Feldern, und der Interpreter-Dispatch pro Iteration bestimmt die Wall-Time. Das Ergebnis ist eine Laufzeit von 2,5 Stunden für 10 000 Abfragen bei optimaler Thread-Anzahl auf dem 5-Millionen-Histogramm-Workload `c256_56gb`.

Diese Arbeit reimplementiert Fainders Abfragephase in Rust und nutzt die resultierende Engine als empirisches Instrument, um zu charakterisieren, was die Leistung auf moderner Mehrkern-Hardware wirklich bindet, sobald der Python-Overhead entfernt ist. Ablationen über mehr als zwanzig Optimierungsachsen auf einem Sapphire Rapids (Xeon 8468H) Server ergeben zwei strukturelle Ergebnisse. Erstens ist verteilungsbasierte Suche auf dieser Plattform durch **vier verschiedene Leistungsobergrenzen** begrenzt—Ein-Thread-Rechenzeit / L1-Latenz, gemeinsame L3-Kapazität, DRAM-Bandbreite auf Socket-Ebene und der Cross-Socket-UPI-Interconnect—jede bindet in einem anderen Thread-Bereich und wird von einer eigenen Klasse von Interventionen adressiert. Zweitens komponieren sich Optimierungen, die einzeln eine dieser Obergrenzen angehen, *negativ*: fünf unabhängige Ablationen auf fünf verschiedenen Achsen zeigen dieselbe Form, wobei eine gröbere Optimierung die bindende Obergrenze am Kompositions-Baseline bereits erreicht hat und der Overhead der feineren Optimierung durch Buchhaltung, Dekodierung oder Koordination als Netto-Regression statt als abnehmender Ertrag erscheint.

Die Rust-Engine erzielt eine 500–725×-Beschleunigung in der Suchphase und

eine 524–598×-Ende-zu-Ende-Beschleunigung gegenüber der Python-Baseline bei maximalem Durchsatz auf präzisions- und recall-erhaltenden Ausführungen über drei Datensatzgrößen; weitere 11–13% können bei $t \leq 48$ durch Bindung des Prozesses an einen einzelnen NUMA-Knoten zurückgewonnen werden. Der ebenfalls nach Rust portierte Rebinning-Kernel trägt eine separate $\sim$18×-Beschleunigung in der Ausrichtungsphase der Indexkonstruktion bei. Diese Zahlen sind eine nachgelagerte Konsequenz der obergrenzenbewussten Neuentwicklung und nicht der primäre Beitrag dieser Arbeit.

Der methodische Hauptbeitrag ist die **Pre-Flight-Obergrenzenidentifikation**: bevor eine Optimierung implementiert wird, sondiert man die Zielkomposition mit Hardware-Performance-Countern und schließt jede Optimierung aus, deren Zielachse dort nicht mehr bindet. Das Muster wird für jede Perzentilprädikat-Abfrage-Engine erwartet, deren Zugriffsmuster eine Kette von Binärsuchen pro Cluster ist, und allgemeiner für hardwarenahe Systeme mit gestapelter Obergrenzenstruktur.

Abstract

Distribution-aware dataset search addresses the challenge of discovering datasets by their statistical properties, in settings where the raw data is not accessible and only per-column histograms are available. Fainder [2] is a recently proposed index for this problem: it clusters histograms with K-means, aligns them onto a shared bin grid inside each cluster, and precomputes cumulative-density arrays so that percentile predicates can be answered by binary search rather than a full histogram scan. On the GitTables corpus of five million histograms it achieves more than two orders of magnitude speedup over a naïve profile-scan baseline while shrinking the index from over 1 TB to 3.2 GB. The reference implementation is written in Python, and the Python overhead is what dominates its query time: the Global Interpreter Lock serialises the per-cluster loop across threads, the NumPy Array-of-Structs layout pollutes cache lines with unread hist-id fields, and per-iteration interpreter dispatch takes over the wall time. The result is a 2.5-hour runtime for 10,000 queries at optimal thread count on the 5M-histogram `c256_56gb` workload.

This thesis re-implements Fainder's query phase in Rust and uses the resulting engine as an empirical instrument for characterising what actually binds performance on modern multi-core hardware once the Python overhead is out of the way. Ablations across more than twenty optimisation axes on a Sapphire Rapids (Xeon 8468H) server yield two structural findings. First, distribution-aware search on this platform is bounded by **four distinct performance ceilings**—single-thread compute / L1 latency, shared L3 capacity, on-socket DRAM bandwidth, and cross-socket UPI interconnect—each binding at a different thread-count regime and each addressed with a distinct class of intervention. Second, optimisations that individually address one of these ceilings can compose *negatively*: five independent ablations on five different axes exhibit the same shape, a coarser optimisation having already collected the binding ceiling at the composition baseline, and the finer optimisation's bookkeeping, decode, or coordination overhead emerging as a net regression rather than a diminishing return.

The Rust engine delivers a 500–725× search-phase speedup and a 524–598× end-to-end speedup over the Python baseline at peak throughput on precision- and recall-preserving executions across three dataset scales; a further 11–13% can be recovered at $t \leq 48$ by pinning the process to a single NUMA node. The rebinning kernel, also ported to Rust, contributes a separate $\sim$18× speedup on the alignment phase of index construction. These figures are a downstream consequence of the

ceiling-aware re-engineering, not the thesis's principal claim.

The principal methodological contribution is **pre-flight ceiling identification**: before committing to an optimisation, probe the target composition with hardware performance counters and rule out any optimisation whose intended axis no longer binds there. The pattern is expected to hold for any percentile-predicate query engine whose access pattern is a chain of per-cluster binary searches, and more broadly for hardware-conscious systems with a stacked-ceiling structure.

Acknowledgments

For recommendations on writing your Acknowledgments see [25].

Contents

List of Figures

List of Tables

List of Abbreviations

DIMA Database Systems and Information Management

List of Algorithms

1 Introduction

The surge of popularity of open data repositories, scientific data lakes, and enterprise data catalogs (among others) has presented us with the challenge of discovering datasets, in cases where access to the raw data is not practical, or even restricted in some cases. Depending on the data vendor, such as one that may distribute privacy-sensitive information, we are only provided via a statistical data summary in the form of the metadata. Distribution-aware dataset search [2] addresses this by developing a way for queries to be done over statistical properties of datasets, instead of the aforementioned raw data. It does so by using percentile predicates in the form *"find all columns where the p-th percentile exceeds threshold t"*.

Fainder [2] introduces an index structure for this exact task. A collection of histograms is created using K-means clustering, it then aligns each histogram that is attached to a cluster onto a shared bin grid, it then precomputes cumulative density arrays that enable us to answer percentile predicates via binary search, instead of needing to traverse histograms fully. On the GitTables benchmark of five million histograms, Fainder achieves more than two orders of magnitude speedup over a naïve profile-scan baseline while reducing memory footprint from over 1 TB to 3.2 GB.

This thesis is about accelerating Fainder. More explicitly, the goal is to re-implement Fainder's query phase in Rust and to understand what actually binds its performance on multi-core hardware used in modern machines. The investigative portion is the crux of this thesis: it uncovers four major binding ceilings on the Sapphire Rapids Xeon, and shows a pattern of *negative composability* across five different optimisation axes. Lastly, we also have a pre-flight ceiling-identification methodology that was developed from that pattern. All these findings are based on the access pattern that Fainder is implemented on, which is per-cluster binary search over respective density arrays. Fainder here is the empirical measuring stick — so to speak. The findings throughout this thesis will also apply to any other percentile-predicate query engine with that type of access pattern, which makes the methodology here portable beyond its codebase.

1.1 Motivation

Fainder's original implementation [2] was implemented in Python, and uses NumPy. Using Python, means that Fainder's code is subject to the Global Interpreter Lock (GIL) [19] which is responsible for serialising bytecode across threads in a single process. Due to the GIL, multi-threading the cluster-chain loop cannot reduce wall-clock query time. The reference implementation also makes use of NumPy using the Array of Structs (AoS) layout for structured arrays, which is not suitable for Fainder's access pattern, due to it only reading the percentile-value column, and ignoring its *hist id* field on every comparison. It shows the degradation of L1 hit rate [14] through the pollution of the cache-line. Per-iteration interpreter dispatch over the entire cluster chain compounds onto both.

All of this results in the query time of the reference's implementation running in the interpreter, not in the binary search algorithm it revolves around. The experiments on this platform uses a two-socket Sapphire Rapids 8468H server with 96 physical and 192 logical cores (Read more about the Experimental setup at §**??**). Python-based fainder has a 2.5 hour runtime to process $10,000$ queries on the $\sim$5M-histogram `c256_56gb` workload with a thread count that minimizes its wall (This has been extrapolated from a 1 K-query subsample, see §**??**). If a throughput floor of hours per workload is needed, it weakens the viability for data exploration.

1.2 Research Challenge

The challenge in this thesis is to address and attempt to close the gap between Fainder's algorithmic potential, and the performance limits set by its Python implementation, without sacrificing the correctness guarantees that its existing use depends upon. The deeper and more investigative challenge, is the characterisation of *the bounds that are present once the Python overhead is removed*, which hardware ceilings bind, and under which regime, and how the ablations targeting one ceiling, work with another.

To perform this research, we need to perform systematic low-level profiling in order to establish and identify the dominant bottlenecks of performance, design a replacement query engine which is able to make use of cache-conscious memory layouts and the parallelism provided by the hardware. Lastly, we would also need to validate that each of these implementations produces *bitwise* identical results to the original reference implementation across all workloads. It is also vital that the solution of this research integrates into the current Python implementation and ecosystem, since the rest of Fainder's core (like clustering and data-loading) is efficiently implemented in Python. The rebinning phase of the index construction has

been ported to Rust in Chapter ??, but the K-means and histogram construction remain implemented in Python.

1.3 Four Performance Ceilings on Sapphire Rapids

The main empirical finding of this thesis (as unearthed by our ablations across more than twenty optimisation axes) is that distribution-aware search on our hardware setup, a Sapphire Rapids (Xeon 8468H) server, is subject to binding by **four performance ceilings**, each having a distinct binding under a different regime, each addressed with a class of intervention.

(i) Single-thread compute / L1 latency. When we have a thread count of $t \leq 8$, per-cluster column slices fit into $L1d/L2$, and the performance wall here is formed by the latency chain of dependent loads in the `partition_point`. Out of five latency-end approaches (Leaf-Stage AVX-512, MSHR-pipelined batch search, horizontal-SIMD lockstep, PGM learned index, branchless k-ary), all of them measure within $\pm 7.3\%$ of the default at every $(\text{dataset}, t)$ for every $t \geq 8$. Leaf-Stage AVX-512 achieves a 6.7% speedup at $t = 1$, since the load-latency chain is the only active ceiling. That is evidence that the grunt of the cost is the *number* of dependent chains (one per cluster), not the depth or throughput of any individual chain (which scales as $\log_2$ of the bins per cluster), and that SIMD is only useful in a regime where no ceiling has already collected the wall budget (§??).

(ii) Shared L3 capacity. At thread counts in the intermediate range ($t \approx 16$) the working sets of each thread begin to overlap in the shared L3 (105 MB on SPR). One optimisation is the reduction of the footprint of per-cluster bytes, which is done via `f16` halving the per-cluster bytes. This optimisation contributes inside the composed bundle, but does not gain standalone at `c256_56gb`. Its contribution is made clearer only when `pooled` first removes the allocator contention, which dominates the performance wall. On the other hand, optimisations that *expand* the footprint (Array-of-Structs padding, naïve `horizontal-simd` gather lanes) are at a loss at intermediate t (AoS is +16.5% slower at $t = 8$), and flip to a small $\sim 4\%$ gain at $t \geq 64$ as ceiling (iii) takes over. The hierarchy of all binding ceilings is shown in §??. Morsel is designed to spread the cost of cold-loading a cluster across workers so the full L3 miss cost isn't burdened on one single worker. In this case, the miss cost is not the binding ceiling, so all morsel's coordination bookkeeping shows up in the wall time with nothing to offset it, and it comes out slower as a net result (§??).

(iii) On-socket DRAM bandwidth. At thread counts in the higher range ($t \in [32, 48]$) our ceiling becomes the per-socket DDR5 bandwidth. The optimisation that most directly tackles this ceiling is the `packed-ids` emit-phase ID packing, which reduces the bytes written per emit (and subsequently the demand on the memory controller). Standalone it achieves a small but consistent $\sim$ 2–5% gain at $t \geq 32$. Through search-phase byte reduction, `f16` contributes, but as noted in ceiling (ii) it wins only *inside* the composed bundle, saving $\sim$ 21% at $t = 16$ on `c1024_56gb`, and does not achieve a standalone speedup on `c256_56gb` at 10-rep resolution (§??).

(iv) Cross-socket UPI interconnect. At the high thread count of $t > 48$ which requires both sockets, the Ultra-Path Interconnect (UPI) between the two NUMA nodes becomes a fourth ceiling. We can remove 11–13% of the wall time at $t \leq 48$ by single-socket placement (`numactl --cpunodebind=0 --membind=0`). If we interleave the memory across sockets (`numactl --interleave=0,1`), we disrupt the hardware prefetcher and regress 26–42% on `c256_56gb` and 33–42% on the out-of-distribution density point `c1024_56gb`. We reproduce this negative composability on the memory-locality axis across density (§??).

These four ceilings are what Chapter ?? works through with evidence, they are named here so that the ablation chapter reads as *evidence for each ceiling*, instead of seeming like a collection of unstructured optimisation attempts.

1.4 Negative-Composability as a Structural Finding

The characterisation of four ceilings is the thesis's principal finding on the *empirical* side. On the other hand, the *structural* finding is that the ceilings are not independent: they interact in a way that makes the composition of optimisations across ceilings non-trivial. The five ablations that establish this pattern are:

- emit-phase ID packing over `bestofsuite` (§??),
- K-batch cluster amortisation over `f16` (§??),
- morsel-driven phase coordination over query-batch (§??),
- per-cluster reindexing over packed-ids (§??), and
- page-granular NUMA interleave over first-touch placement (§??).

All these five ablations exhibit the same shape:

A coarser optimisation has already addressed the current binding ceiling at the composition baseline. The finer optimisation's overhead finds no more headroom on the ceiling it was designed to tackle, and ends with a net regression rather than diminishing returns.

The pattern that emerges here does not show diminishing returns at the margin, but rather a sign-flipping at the margin. Ablation (ii) shows a +7% regression at the cross-worker case (§**??**), and ablation (iv) shows a +244% regression at the decoder-access-pattern case (§**??**). Each case has an argument for positive composition, which is then falsified by the `perf` counters.

pre-flight ceiling-identification is the methodological consequence of this pattern: before committing to implement a finer optimisation, we can use perf counters to probe the binding ceiling at the *target composition*, if it's not the ceiling that it was designed to tackle, then a regression is likely to occur. We demonstrate five independent instances of this pattern in §**??**, and synthesise the pattern in §**??**.

1.5 Dispatch-on-Regime

The main findings in this Thesis are the four performance ceilings, and negative composability. This leads to developing a dispatch policy based on the regime that the workload is in. It selects a "best in regime" combination of build features and deployment flags. On the 18-cell calibration grid it picks the empirical winner in 17 cases and lands within 5% in all 18, on the out-of-distribution `c1024_56gb` density point it holds all 6 regime assignments (§**??**, §**??**).

We do not consider the dispatch artefact as the thesis's main scientific contribution, but rather as the engineering consolidation of the ceiling characterisation.

1.6 Contributions

The thesis's contributions are organised as follows:

(1) **Rust re-implementation of Fainder's query pipeline.** A Rust implementation of Fainder's query engine, preserving the index structure (K-means clustering, per-cluster rebinning, cumulative-density arrays) and interface compatibility with the Python API using the PyO3 bindings. Behind the curtain, Rust has a SoA memory layout, Rayon-based work-stealing parallelism, and a cache-conscious design that is aware of the four ceilings. We validated the engine to produce bitwise-identical results to the Python reference implementation across all workloads (§**??**). This implementation delivers a 500–725× speedup in the search phase over the baseline Python implementation

at $t \geq 16$. At single thread ($t = 1$), where our ceiling-aware engineering has less room to compose, since only one ceiling (ceiling (i)) is active, the speedup is 114–129×. With result serialisation, the end-to-end speedup is 524–598× when throughput is at its peak ($t \in \{16, 96\}$), meaning that the result-boxing (which is done in Python) is negligible compared to the query cost that the Rust engine displaces (§**??**). Additionally, there is a gain of 11–13% at $t \leq 48$ when the deployment-time NUMA placement is used (§**??**). The Rust re-binning kernel achieves a separate $\sim 18\times$ speedup on the index-construction alignment phase (§**??**). The engineering is presented in Chapter **??** as our main driver for the ablations. The four-ceiling identification, the pre-flight identification methodology, and the negative-composability pattern are the thesis's principal findings.

(2) **Four-ceiling micro-architectural characterisation.** Identification of four distinct performance ceilings via `perf` counters on the Sapphire Rapids (Xeon 8468H) server. Each of these ceilings binds under a regime that is addressable through a specific intervention. The evidence for the ceilings (Section 1.3) is presented in Chapter **??**.

(3) **Negative-composability pattern.** Five independent instances on five different axes of the same compositional shape, leading us to the result of negative composability of optimisation. This shows as a structural property of the access pattern rather than an incident of any single ablation (Section 1.4, evidence and mechanism are presented in §**??**, and synthesis is done in §**??**).

(4) **Pre-flight ceiling-identification methodology.** We identify binding ceilings at the *target composition* using data from `perf`. These help us to avoid implementing an optimisation that is expected to be positive, but is actually negative at the composition baseline. The methodology is presented in §**??**.

(5) **Dispatch-on-regime deployment recipe.** An automatic dispatch with a selection of build features and deployment flags. Validated 17/18 exactly and 18/18 within 5% on the calibration data scaled (Section 1.5, implemented within: `fainder/execution/dispatch.py`, derived in §**??**, OOD validation in §**??**).

The engineering artefacts (1 and 5) are the empirical staging ground for the scientific and methodological findings (2, 3, and 4). We would not have been able to identify the four ceilings without the Rust engine, and we would not have been able to identify the negative composability pattern without the ablations. The pattern itself, not the speedup that the Rust engine achieves, is what generalises beyond this codebase.

1.7 Thesis Structure

The remainder of this thesis is organised as follows:

1. Chapter 2 provides the scientific background on distribution-aware dataset search, the Fainder index structure, and the hardware-conscious optimisation techniques applied in this work.

2. Chapter **??** documents the experimental setup (hardware, software, datasets, queries), the Rust implementation architecture, the measurement protocol, and the a-priori hypotheses each experiment tests.

3. Chapter **??** reports the experimental evaluation as evidence for each of the four ceilings identified in Section 1.3, with the negative-composability pattern in its closing section.

4. Chapter **??** surveys related work, with particular attention to the composability of optimisations in classical query optimisation and to the joint design of algorithms and hardware.

5. Chapter **??** concludes and outlines directions for future work, including the experiments most likely to confirm or extend the structural finding.

6. Appendices **??**–**??** provide additional information on the experimental setup, the Rust engine's implementation, and the complete set of experimental results.

2 Scientific Background

This chapter serves as an overview for readers unfamiliar with the topics. It is a comprehensive survey of prior work and hardware concepts this thesis builds on.

- Section 2.1 introduces distribution-aware dataset search and the Fainder index.
- Section 2.2 reviews the hardware and systems-programming material for Chapter **??**
- Section 2.3 is a short primer on what it means for two hardware-conscious optimisations to compose, which is dived into in Chapter **??**.

A reader comfortable with these topics can jump to Chapter **??**.

2.1 Distribution-Aware Dataset Search

2.1.1 Data Discovery and Metadata-Only Access

Open data lakes and data collected in corporate catalogs are usually in the range of having millions of columns. Data scientists with a particular statistical profile of data in mind (for example, columns with a median between 50 and 100, or data in which the 90th percentile has a certain cutoff) would usually need to scan every file. For federated and privacy-sensitive data, it is usually common that the raw data is not readable at all, and the only provided evidence is the statistical metadata such as per-column histograms that are computed before access is restricted [2].

This is a retrieval problem in distribution-aware dataset search. Under a collection of histograms $\mathcal{H} = \{h_1, \ldots, h_n\}$ which summarize individual columns, and given a query $\langle p, \bowtie, \tau \rangle$ which returns every histogram h for which the p-th percentile of h satisfies $h[p] \bowtie \tau$, the system must return the matching subset without ever touching the underlying rows [2].

2.1.2 Fainder Index

Fainder [2] is a three-phase index that exists for this type of query.

Index construction. In this phase, we group n histograms into k clusters by using K-means on the histogram vectors. In each cluster, we align the histograms onto a shared bin grid. Fainder makes use of two alignment strategies: *rebinning*, which projects each histogram onto a fixed bin count, which is compact and fast, with a small alignment error near bin boundaries; and *conversion*, which produces two index variants per cluster (a rebinned variant plus a higher-accuracy converted variant), at $\approx 2\times$ the memory cost and a 10–15% slower query. In this thesis we use rebinning throughout the entire work, since the accuracy delta stays the same throughout all of our findings.

Index structure. *FainderIndex* is a Union of all *SubIndex*'s. Each cluster c has a SubIndex that stores the cumulative density values of all histograms in c at each bin boundary b. The cumulative density values are stored in a sorted array, along with the matching histogram identifiers.

Query execution. To execute a query $\langle p, \bowtie, \tau \rangle$, the engine traverses through each cluster, where it searches the shared bin grid via binary search in order to find the bin b^* that corresponds to the reference value τ. Then, it performs another (binary) search on the cumulative density array at boundary b^* to collect the saved histogram IDs that follow the percentile predicate. The $\cup$ across all clusters is the final answer.

The query phase can be thought of as a sequence of binary searches, of one per cluster. The way the query phase works, is the reason this thesis concerns itself with cache- and memory hierarchy effects, rather than e.g. SIMD throughput on a scan that is streaming.

2.1.3 GitTables and the Evaluation Datasets

Both the original Fainder paper and this thesis make use of the *GitTables* [9] corpus, which is a public collection of over five million tabular columns made up of GitHub CSV repositories.

From the GitTables corpus we derive four datasets to evaluate in Chapter **??**. The derivation process is as follows:

1. All non-numeric columns are filtered out, and any columns with ≤ 100 rows are removed, leaving us with 5,017,619 histograms.

2. Using K-means, histograms are clustered into $k = 256$ clusters, and all histograms are rebinned to a fixed bin count of 256, producing the `c256_56gb` dataset.

3. Subsequently, we use clustering again to cluster the histograms into $k = 1024$ clusters again, and rebin each histogram into a bin count of 1024. This produces the `c1024_56gb` dataset.

4. We sample 10,000 queries from the query workload used in the original Fainder paper, and we store them in a file that is shared across all four datasets.

5. Lastly, to create `c256_30gb` and `c256_10gb` datasets, we sample 30% and 10% of the histograms from the `c256_56gb` dataset, and re-do the K-means clustering and rebinning to $k = 256$ clusters and a fixed bin count of 256.

In the naming convention, the first number is the K-means target k, and the second number is the approximate size of the dataset in gigabytes. The four datasets are summarized in Table **??**.

The query workload is a fixed set of 10,000 *(percentile, op, threshold)* tuples shared across all four datasets via symlink. Queries are dataset-independent: no column IDs appear in the query schema.

Note that the amount of clusters k is not the amount of clusters that are actually used in the index. Some clusters may be empty, and some clusters may be merged if they are too small. The actual number of clusters is reported in Table **??**, which are the "effective clusters" that are used in the index.

2.2 Hardware-Conscious System Design

In this section we cover the hardware and systems-programming background that Chapter **??** builds on. The goal of this section is to be a reference point for each of the ceilings introduced in Chapter 1. This also serves as a reference for the systems programming techniques used in the Rust implementation of Fainder, which is described in Chapter **??**.

2.2.1 Modern Superscalar CPU Architecture

Modern server CPUs are out-of-order superscalar processors. The ancestor of these processors is Tomasulo's 1967 dynamic-scheduling algorithm [23]. In the CPU, each core fetches and decodes instructions in the front-end, dispatches them to a reorder buffer (ROB) which has 200–350 in-flight operations. Each core also has several (execution) ports that are able to perform multiple operations per cycle [8, 6]. Most cores retire up to four scalar instructions per cycle, which is the theoretical ceiling for instructions-per-cycle (IPC) [10].

Instructions per cycle (IPC). IPC is $\frac{\text{retired instructions}}{\text{cycles}}$. If a program is scalar and serial, and never stalls, it can approach IPC ≈ 4 on modern hardware. The following three stall categories are the main reasons IPC is lower than 4 in practice:

1. *Front-end stalls*: The instruction fetch mechanism can't keep up, this usually happens after a branch misprediction (Section 2.2.6).

2. *Back-end stalls*: the execution ports are fully utilized, this could be due to a lack of independent instructions to fill the ports, or because a dependency chain serializes the work.

3. *Memory stalls*: a load waits for data. L1 hit takes 4–5 cycles, L2 hit 12–15, L3 hit roughly 40, DRAM 150–300.

Work by Yasin [28] breaks the stall cycles into these categories. IPC ≈ 4 (nearing the upper end) indicates work that is compute-bound with healthy instruction-level parallelism, and an IPC ≈ 1 means that the work is either heavily dependent or memory-bound.

Out-of-order execution and the ROB. If a cache miss occurs and causes a load to stall, independent instructions are dispatched by the ROB, that effectively overlap the miss latency. Modern CPUs make use of this technique to tolerate slow memory. The scenario where this cannot be used is when a load is *dependent* on the one before it, so load $i+1$ cannot be issued, until load i completes. Binary search is a prime canonical example for this, where each step reads `arr[mid]`, and the next `mid` depends on the current comparison result, thus breaking the out-of-order execution mechanism on modern CPUs.

2.2.2 Memory Hierarchy and Cache Efficiency

There are three layers of caches that sit between the core and DRAM, hierarchically:

- **L1 cache** is the smallest and fastest, typically tens of KB per core. It has the lowest latency, around 4–5 cycles for a hit.

- **L2 cache** is larger, typically a few MB per core. It has a higher latency, around 12–15 cycles for a hit.

- **L3 cache** is the largest and slowest, typically tens of MB shared across cores on the same socket. It has a higher latency, around 40 cycles for a hit.

Above all of these sits the DRAM [8, 5]. A cache line (which is essentially a block of memory that is transferred between the cache and DRAM) is typically 64 bytes wide. Manegold et al. looked at cache-conscious analysis for database operators [14].

Figure 1 demonstrates the access latencies of each level of the memory hierarchy on the platform used for this thesis. There are dotted vertical cutoffs that indicate the working-set size at which we involve the next, slower level of the hierarchy. The x axis is logarithmic, with the unit of nanoseconds, since we span six orders of magnitude from $\sim 1\,\text{ns}$ to $\sim 5\,\text{ms}$. Due to the shift in order of magnitude, we

can observe measurable wall-time changes when we shift a working set from L3 into DRAM, but any experiment that bottlenecks on disk-backed storage will be dominated by the disk latency, and the cache hierarchy will be largely irrelevant.

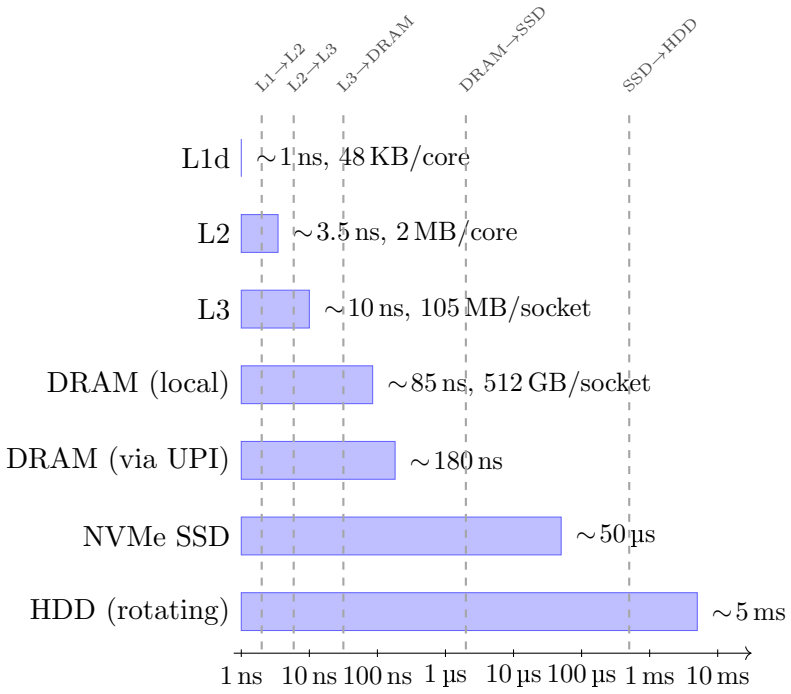

Figure 1: Access latencies along with the capacity of each level of the memory hierarchy on the Sapphire Rapids (Xeon 8468H) platform. X axis is logarithmic, with dotted lines along the Y axis indicating the working-set size at which the next slower level takes over (approximate transitioning zones, not exact cutoffs). The latencies illustrated are approximate values for warm access, not peak throughput. Exact numbers may vary depending on prefetcher state, memory controller queueing, and NUMA policy.

Hardware prefetchers. Multiple prefetchers are attached to each core, they are responsible for monitoring access patterns and pulling cache lines ahead of demand. L1 stride prefetchers catch sequential and constant-stride accesses (which are common in linear scans). L2 streamers catch more coarse patterns. Neither can predict the data-dependent addresses of binary-search, so binary search and other pointer-chasing or indirect-lookup workloads see little prefetcher help.

MSHRs. It is possible for a core to experience multiple cache misses in-flight at once, ranging from 12–20 misses per core on Intel Skylake and later. Independent loads that all miss L1 perform round-trips to DRAM in parallel, which amortises

the latency. A dependent load is unable to perform such a thing, since the address of the next load is not known until the previous one is returned.

DRAM bandwidth. There is a fixed bytes-per-second budget for each DRAM channel. The Xeon 8468H that is used in this thesis has eight DDR5-4800 channels per socket, giving it a per-socket peak of $\sim 300\,\mathrm{GB/s}$, and the two sockets together provide $\sim 600\,\mathrm{GB/s}$. When there's an influx of data streamed from multiple cores, that budget becomes contended, and the effective per-core bandwidth drops. This differs from DRAM *latency*, which is a round-trip time of one load, and the bandwidth only becomes binding when the aggregate traffic across all cores approaches the channel capacity.

Why this matters for Fainder. The main hot path in Fainder is its binary search over a sorted column of cumulative density values. Using an SoA layout, the cache lines loaded will only contain relevant data. With an AoS layout, those same lines will contain half values, half histogram-IDs, which the search phase never reads. The pollution of cache lines effectively halves the values-per-line ratio. Chapter **??** shows this layout effect is measurable but small once larger ceilings dominate at scale.

2.2.3 NUMA and the Cross-Socket Interconnect

Servers with multiple sockets split their DRAM into per-socket *NUMA nodes* [13], Each of these sockets reaches its local node at normal DRAM latency, but reaching the other socket's node at an additional 50–150 cycle cost, via Intel's Ultra-Path Interconnect (UPI) link between the two CPU packages. The NUMA performance pathologies that we see in multicore workloads when the allocations and threads end up on different nodes are catalogued by McCurdy and Vetter [17].

This thesis uses a two-socket platform with 96 physical cores, and 48 cores per socket. The default Linux scheduler does not constrain a thread to a single socket, so any thread count above 48 will span both NUMA nodes out of necessity, which leads to cross-socket traffic. `numactl`'s `--membind` and `--cpunodebind` flags constrain a process to one node, which removes UPI traffic, but halves the core budget.

For our fourth ceiling in Chapter **??** we use the cross-socket interconnect as our source of contention. Both UPI and on-socket DRAM fall under the label "memory bandwidth", but are binding under different conditions, and are measured differently. UPI has an empirical penalty that is distinguishable from the penalty for saturating local DRAM. The NUMA placement experiment of Chapter **??** separates the two.

2.2.4 Structure of Arrays vs. Array of Structs

SoA vs AoS is a classical concern when designing data-oriented systems [14]. Figure 2 illustrates the differences between them:

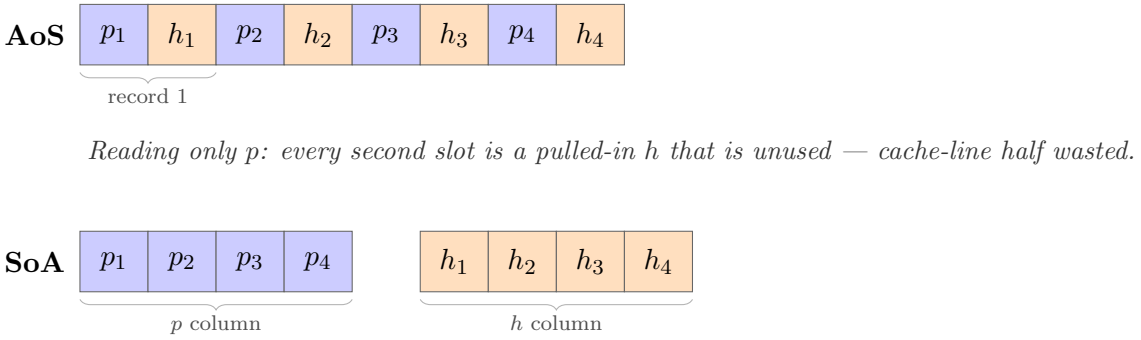

Figure 2: AoS interleaves fields per record and SoA stores each field in its own contiguous array. For Fainder's access pattern (which touches only the percentile-value column (p) on every comparison), SoA halves the cache-line footprint by avoiding the unused hist-id (h) lanes.

SoA wins on cache utilisation when the access pattern is field-specific. For Fainder, only the percentile-value field is read during the search, and the histogram-ID is needed after the matching range has been located. SoA has about double the useful values per cache line. There is a $\sim 16\%$ speedup at $t = 8$ on `c256_56gb` (with smaller gains of 2–3% on the smaller `c256_10gb` and `c256_30gb` datasets), but this advantage becomes less relevant once the limiting ceiling becomes DRAM bandwidth at ~ 48 threads.

2.2.5 Eytzinger BFS Layout

During binary search, sorted arrays are accessed in a pseudo-random pattern, which is bad for cache locality. There are unpredictable strides that make the hardware prefetcher ineffective, and each access hits a new cache line. The *Eytzinger BFS* (breadth-first) layout reorders a sorted array, such that the binary search tree is stored breadth-first. It stores the root at index 0, its children at 1 and 2, their children at $3 \ldots 6$, and so on [12]. We can store the first four levels of that tree in a single 64-byte cache line, that means that the first four bisections of an array cost one line load, instead of four. The L1 stride prefetcher can help with subsequent levels, since they access consecutive positions $8, 9, 10, \ldots$ instead of scattered, random ones. Cache-oblivious analysis of layouts of this kind is attributed to research by Brodal and Fagerberg [4].

For searches of length n, this layout replaces $\log_2 n$ dependent-load stalls at pseudo-random offsets with a shorter sequence of loads at predictable stride, in addition it adds a bit-reversal permutation to convert the final BFS index back to the sorted-position index. This technique accelerates search when the cache is cold (when the searched array is not already in L1), and its cost is a small memory overhead. Benchmarks by Khuong and Morin [12] show wins for Eytzinger at large n cold-cache searches, and small losses at small n where the binary search already fits in L1 after a miss. In Fainder's per-cluster columns, Eytzinger's cache-cold advantage largely disappears, since the columns are L1-resident after the first cluster access in a query batch. Chapter **??** explores whether Eytzinger remains useful in the presence of thread-count-driven eviction.

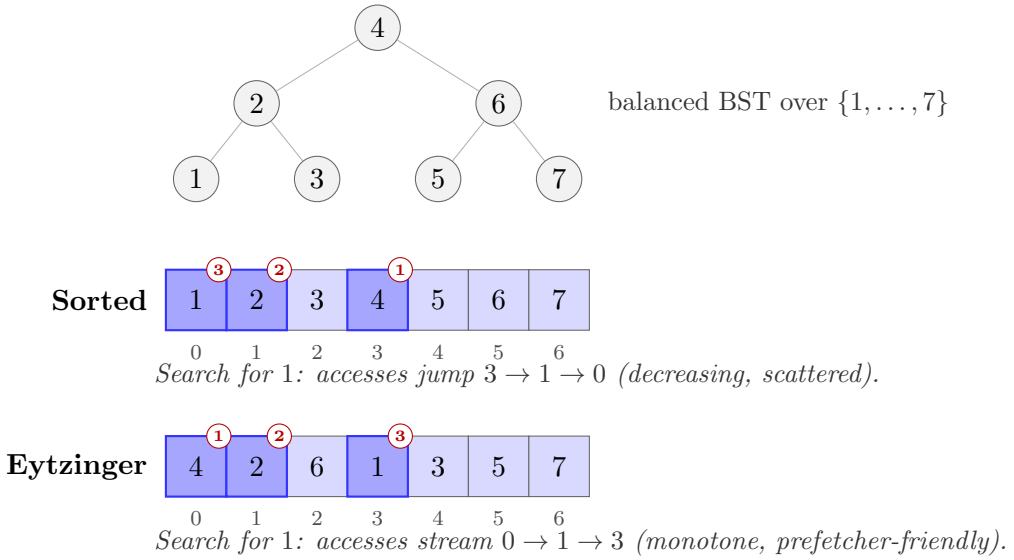

Figure 3: Sorted vs. Eytzinger (BFS) layout for a 7-element array. The Eytzinger layout stores the balanced BST in breadth-first order, so the root sits at index 0 and the early bisection nodes cluster in the low-index prefix. Binary-search accesses then form a prefetcher-friendly sequence instead of the pseudo-random jumps of the sorted layout.

2.2.6 Branch Prediction and Branchless Code

The front-end of a CPU pipeline fetches and decodes instructions. When a CPU encounters a conditional branch (for example an if statement), it has to decide which path it should follow before the conditional is evaluated, in order to prevent pipeline stalls. Modern CPUs employ branch prediction algorithms in order to predict the outcome of a branch; the branch predictor learns the history from

previous executions [29] and predicts whether a branch will be taken or not. If the branch predictor guesses correctly, then the execution continues without interruption, however, if the prediction is incorrect, then the pipeline gets flushed by the CPU and the execution has to restart from a correct path, which is a significant performance penalty. This may cost 15–25 cycles, depending on the depth of the pipeline and the specific architecture.

CMOV. is the *Conditional move instruction* which is a branchless alternative to conditional branches. It selects between two register values based on a flag without emitting a branch, so misprediction is not possible. Standard binary-search routines in Rust (`slice::partition_point`), C++ (`std::lower_bound`), and NumPy's C core (`numpy.searchsorted`) compile to a CMOV-based branchless loop. The following pseudocode illustrates this:

```
while size > 1 {
    half = size / 2;
    mid  = base + half;
    // CMOV: conditional update of base, no branch
    base = (arr[mid] < target) ? mid : base;
    size -= half;
}
```

This shows that the cost is not branching, but the chain of dependent L1 loads. Each iteration depends on the result of the previous comparison. For a column of $n \approx 1{,}400$ elements the chain is $\log_2 n \approx 11$ sequential L1-latency loads. This amounts to about 50 cycles per cluster search, and multiple thousand cycles per query. In Chapter **??** we confirm that the compiler emits the branchless loop, with a measured branch-misprediction rate of 0.01–0.03%.

2.2.7 Single-Instruction Multiple-Data (SIMD)

SIMD allows a single instruction to operate on multiple data elements in parallel [11]. Modern x86 CPUs expose SIMD extensions, which are used to accelerate vectorizable workloads. The most relevant SIMD extensions for this thesis are AVX2 and AVX-512. AVX2 employs 256-bit vector registers, and AVX-512 employs 512-bit vector registers. A single `vcmpps` is comparing eight `f32` values against a scalar that is broadcast and writes an 8-bit mask in one execution slot. Zhou and Ross demonstrated systematic SIMD use in database operators [30], later work scaled SIMD to column scans [27] and to binary search [21].

Two SIMD patterns are relevant to binary search:

1. *SIMD within a search.* The last steps of binary search with fewer than eight elements remaining, can be vectorized. Those remaining elements are

loaded into a vector register, compared against the target, and a bitmask is generated to locate the partition point. This reduces the number of iterations and dependent loads, improving performance when the data is cache-resident.

2. *SIMD across searches.* If we have multiple independent, parallel binary searches, the B-th steps of each search can be done in tandem. Since these loads do not depend on each other, MSHRs can pipeline them into parallel DRAM round-trips. Theoretically, this can hide up to 8× of memory latency, but only if the bottleneck is actually memory latency.

Chapter **??** explores and evaluates both of these approaches, and concludes that for Fainder, the L1-resident columns are too small for SIMD to be effective. The first pattern is only relevant for the last few iterations of a search, which is a small fraction of the total search time. The second pattern requires multiple independent searches to be in-flight simultaneously, which is not the case for Fainder's workload. The workload, instead, is bounded by the L1-latency dependency-chain length, which SIMD cannot shorten.

2.2.8 Hardware Performance Counters

Modern x86 CPUs have performance monitoring units (PMUs) for each core, which count low-level events such as retired (completed) instructions, cycles, branch-retired and branch-missed, cache-reference and cache-miss, and many more [10]. Linux's `perf` tool [26] is able to sample these statistics and reports the aggregates. Yasin's [28] method maps the raw counts onto stall categories, as mentioned in Section 2.2.1. Useful derived metrics include:

- **IPC** $= \frac{\text{retired instructions}}{\text{cycles}}$
- **Branch-mispred rate** $= \frac{\text{branch-misses}}{\text{branches}}$
- **L1 miss rate** $= \frac{\text{L1-dcache-load-misses}}{\text{L1-dcache-loads}}$
- **LLC miss rate** $= \frac{\text{LLC-load-misses}}{\text{LLC-loads}}$. An **LLC miss** is a load that misses the last-level cache (L3) and goes to DRAM. The LLC miss rate indicates whether the working set fits in the last-level cache. A high LLC miss rate indicates that the workload is memory-bound and may benefit from optimizations that reduce memory accesses or improve cache locality.

Counter multiplexing (the act of requesting more events than the PMU has hardware slots for) reduces accuracy. Chapter **??** explains how we adapted to work around it.

2.2.9 Multicore Parallelism and Work-Stealing

To exploit multicore parallelism on a CPU, a program needs either explicit threading or a work-stealing scheduler. Python's GIL [19] serialises Python bytecode across all threads in a single process, that means that for CPU-bound Python code, "true" multicore parallelism is not achievable without leaving its interpreter.

Rayon [22] on the other hand, is a Rust library built on Blumofe and Leiserson's work-stealing scheduler [3], and is designed for data-parallelism. Its `par_iter()` decomposes sequential computations into independent tasks and performs dynamic distribution across threads. Load imbalances are automatically handled by work-stealing. Since Rayon is Rust territory, the GIL does not apply.

We can map Fainder's query execution onto Rayon easily, each query reads a shared, read-only index, with no cross-query data dependencies or shared mutable state. So locks, or atomic operations are not needed. Scheduling and load imbalances are handled by Rayon.

2.2.10 Rust and PyO3

Rust [15] is a systems language that gives performance on a level of C through (LLVM-based) native code generation and zero-cost abstractions. Memory and thread-safety are not a concern because of the ownership and borrowing type system. Rust does not possess a garbage collector, as memory is freed in a deterministic manner when its owning scope ends, so there are no GC pauses and tail latencies are predictable.

PyO3 [18] is a Rust crate that provides bidirectional Rust–Python FFI (Foreign Function Interface). To expose a Rust function to Python, it is annotated with `#[pyfunction]`, it is then compiled into a Python-callable extension, that makes it indistinguishable from a native Python function. The Maturin build tool [16] handles compilation and packaging. Other examples of this pattern include DuckDB [20], Polars [24], and Apache Arrow [1], which expose native cores to Python. PyO3's zero-copy buffer uses NumPy's array protocol [7] to share `numpy.ndarray` data with Rust.

2.2.11 Binary Search Optimisation: `partition_point`

Three-way binary search is a common algorithm for finding the partition point in a sorted array. Its first comparison is data-dependent on the sorted-array distribution, so there is usually a poor prediction rate. Rust's `slice::partition_point` takes a single boolean predicate `|x| x < target` and is then compiled to the CMOV-based branchless loop from Section 2.2.6. It does not emit a branch for

the bisection update, and the only branch in this loop is the `while size > 1` terminator, which is highly predictable.

We can semantically equate this function to NumPy's `searchsorted(..., "left")` function, since it produces bitwise-identical results. We measure the misprediction rate of 0.01–0.03%, which confirms that the CMOV compilation reaches the release build.

2.2.12 Experimental Platform

All measurements in this thesis ran on a single shared-memory server:

- CPU: 2× Intel Xeon Platinum 8468H (Sapphire Rapids), 48 physical cores per socket with SMT-2, 96 physical cores / 192 logical threads total. AVX-512F, AVX-512BW, and AVX-512_FP16 supported.
- Caches: 48 KB L1d per core, 2 MB L2 per core, 105 MB L3 per socket (shared within a NUMA node), 210 MB L3 total.
- NUMA: two nodes, one per socket. Node 0 holds logical CPUs 0–47 with SMT siblings 96–143; node 1 holds 48–95 with siblings 144–191.
- Memory: 1 TB DDR5 across both sockets, eight channels per socket, ∼ 300 GB/s per socket / ∼ 600 GB/s aggregate peak.
- Storage: NVMe SSD (dataset loads) and a RAM-backed tmpfs at `/dev/shm` (used for the storage-hierarchy ablation in Chapter **??**).
- OS: Linux 5.15. Compiler: Rust 1.82.0 (LLVM 19). Python: 3.10.

The Rayon thread pool for the thread-count sweeps in Chapter **??** is set explicitly via `FAINDER_NUM_THREADS`. Thread counts up to 48 fit within one NUMA node's physical core budget, and 64 and above span both sockets, which requires UPI traffic. This clearly separates the NUMA experiment of Chapter **??** from the fourth ceiling.

2.2.13 Rust Compiler Optimisations

Once Rust's "release" profile is enabled, the compiler applies a number of optimisations. The most relevant for this thesis are:

- `opt-level = 3`: enables aggressive optimisations, including inlining, loop unrolling, and vectorisation.

- `lto = true`: enables link-time optimisations, allowing the compiler to perform whole-program analysis and optimisations across crate boundaries.
- `codegen-units = 1`: reduces the number of code generation units, allowing for better optimisations at the cost of longer compile times.
- `target-cpu = native`: optimises the generated code for the host CPU architecture, enabling the use of all available instruction sets and features.

The release profile also permits SIMD intrinsics and auto-vectorisation of compatible inner loops; *However*, Fainder's binary-search hot path is a chain of dependent loads that the compiler cannot vectorise across iterations (due to data dependencies), the explicit AVX-512 leaf-stage variant is evaluated separately in Chapter **??**.

2.3 Composability of Hardware-Conscious Optimisations

Chapter **??** finds that five independent optimisations (each of which is individually beneficial), do *not* compose positively. This is a structural finding, and it is the main empirical contribution of this chapter. The pattern of composability is consistent enough that the thesis names it *negative composability* and treats it as a property of hardware-near systems with multiple stacked ceilings. This section introduces what "composing" two optimisations means and why the classical textbook argument for positive composition can fail in the presence of multiple ceilings.

The textbook expectation. We classically view optimisations as independent actions that each save a measurable resource. For example:

- An SoA layout saves cache-line bytes.
- An `f16` representation halves memory-stage bandwidth.
- A bit-packed ID array shrinks the emit-phase footprint.

In the case of the bandwidth-budget, having optimisation A, which reduces bandwidth use by a factor of r_A, and optimisation B, which reduces it by r_B; classically we would expect that applying both should leave $(1 - r_A)(1 - r_B)$ of the original demand. As long as the workload remains bandwidth-bound, the composition is at worst diminishing-returns positive: the gain from B on top of A is smaller than the gain from B alone, but it stays non-negative.

Why this argument can fail. The above argument has an assumption that the binding resource is the same before, and after A is applied. In a system, such as ours, where there are multiple ceilings, this assumption can fail. That means, that the binding constraint can shift to a different ceiling after A is applied, and B may not address that new ceiling. In such cases, the effect of B can be negative, due to non-zero costs on the new binding ceiling, such as decode work, bookkeeping cycles, and coordination overhead. These costs become visible because the bandwidth slack that hid them before has been consumed by A.

Chapter **??** documents the pattern on five different axes. One optimisation collects the ceiling that binds at the bare default, and the other optimisation's overhead emerges as net regression rather than diminishing return. We generalise this as a structural finding: the default outcome when an ablation chooses its target based on what binds at the bare default rather than at the current best composition.

What this implies for methodology. The attempted ablation methodology, which is commonly used for evaluating optimisations, is not sufficient for a system with multiple ceilings. The ablation methodology applies an optimisation, and measures its delta against the baseline. If the delta is positive, it's an accepted optimisation. In our case, it does not account for the fact that the binding ceiling can shift after an optimisation is applied, and that subsequent optimisations may not address the new binding ceiling. The thesis develops a pre-flight ceiling identification methodology in Chapter **??** that uses perf statistics to forecast which ceiling will bind at our target composition, to rule out optimisations that would have negative effect against a constraint the composition has already collected.

This framework is not specific to Fainder, and it is likely to be relevant for other hardware-near systems that have multiple ceilings. The thesis does not claim that negative composability is universal, but it does provide a concrete example of how it can manifest in practice, and how to reason about it.

Bibliography

[1] Apache Software Foundation: Apache arrow: A cross-language development platform for in-memory analytics. https://arrow.apache.org (2016)

[2] Behme, L., Galhotra, S., Beedkar, K., Markl, V.: Fainder: A fast and accurate index for distribution-aware dataset search. Proceedings of the VLDB Endowment 17(11), 3269–3282 (2024)

[3] Blumofe, R.D., Leiserson, C.E.: Scheduling multithreaded computations by work stealing. Journal of the ACM 46(5), 720–748 (1999)

[4] Brodal, G.S., Fagerberg, R.: On the limits of cache-obliviousness. ACM Transactions on Algorithms 2(2), 307–322 (2006)

[5] Drepper, U.: What every programmer should know about memory. https://people.freebsd.org/~lstewart/articles/cpumemory.pdf (2007), red Hat technical report.

[6] Fog, A.: The microarchitecture of Intel, AMD and VIA CPUs: An optimization guide for assembly programmers and compiler makers. https://www.agner.org/optimize/microarchitecture.pdf (2023), technical University of Denmark.

[7] Harris, C.R., Millman, K.J., van der Walt, S.J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N.J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M.H., Brett, M., Haldane, A., Fernández del Río, J., Wiebe, M., Peterson, P., Gérard-Marchant, P., Sheppard, K., Reddy, T., Weckesser, W., Abbasi, H., Gohlke, C., Oliphant, T.E.: Array programming with NumPy. Nature 585, 357–362 (2020)

[8] Hennessy, J.L., Patterson, D.A.: Computer Architecture: A Quantitative Approach. Morgan Kaufmann, 6th edn. (2017)

[9] Hulsebos, M., Deng, Q., Investigators, T.G.: GitTables: A large-scale corpus of relational tables. In: Proceedings of the ACM SIGMOD International Conference on Management of Data. pp. 1–14 (2023)

[10] Intel Corporation: Intel 64 and IA-32 architectures optimization reference manual. https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html (2023)

[11] Intel Corporation: Intel intrinsics guide. https://www.intel.com/content/www/us/en/docs/intrinsics-guide/ (2024)

[12] Khuong, P.V., Morin, P.: Array layouts for comparison-based searching. ACM Journal of Experimental Algorithmics 22, 1.3:1–1.3:39 (2017)

[13] Lameter, C.: NUMA (non-uniform memory access): An overview. Communications of the ACM 56(9), 59–65 (2013)

[14] Manegold, S., Boncz, P.A., Kersten, M.L.: Optimizing database architecture for the new bottleneck: Memory access. vol. 9, pp. 266–277 (2000)

[15] Matsakis, N.D., Klock, F.S.: The Rust language: Memory safety without garbage collection. Communications of the ACM 57(4), 14 (2014)

[16] Maturin Contributors: Maturin: Build and publish Rust crates as Python extensions. https://www.maturin.rs (2024)

[17] McCurdy, C., Vetter, J.: Memphis: Finding and fixing NUMA-related performance problems on multi-core platforms. In: IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS). pp. 87–96 (2010)

[18] PyO3 Contributors: PyO3: Rust bindings for Python. https://pyo3.rs (2024)

[19] Python Software Foundation: The global interpreter lock (GIL). https://wiki.python.org/moin/GlobalInterpreterLock (2024)

[20] Raasveldt, M., Mühleisen, H.: DuckDB: an embeddable analytical database. In: Proceedings of the ACM SIGMOD International Conference on Management of Data. pp. 1981–1984 (2019)

[21] Schlegel, B., Gemulla, R., Lehner, W.: K-Ary search on modern processors. In: Proceedings of the Fifth International Workshop on Data Management on New Hardware (DaMoN). pp. 52–60 (2009)

[22] Stone, J., Matsakis, N.D.: Rayon: A data-parallelism library for Rust. https://github.com/rayon-rs/rayon (2015)

[23] Tomasulo, R.M.: An efficient algorithm for exploiting multiple arithmetic units. IBM Journal of Research and Development 11(1), 25–33 (1967)

[24] Vink, R.: Polars: Fast DataFrame library for Rust and Python. `https://pola.rs` (2020)

[25] Wallwork, A.: English for writing research papers, p. 306. Springer International Publishing Switzerland (2011)

[26] Weaver, V.M.: Linux perf_event features and overhead. In: International Workshop on Performance Analysis of Workload Optimized Systems (FastPath) (2013)

[27] Willhalm, T., Popovici, N., Boshmaf, Y., Plattner, H., Zeier, A., Schaffner, J.: SIMD-scan: Ultra fast in-memory table scan using on-chip vector processing units. Proceedings of the VLDB Endowment 2(1), 385–394 (2009)

[28] Yasin, A.: A top-down method for performance analysis and counters architecture. In: IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS). pp. 35–44 (2014)

[29] Yeh, T.Y., Patt, Y.N.: Alternative implementations of two-level adaptive branch prediction. In: Proceedings of the 19th Annual International Symposium on Computer Architecture (ISCA). pp. 124–134 (1992)

[30] Zhou, J., Ross, K.A.: Implementing database operations using SIMD instructions. In: Proceedings of the ACM SIGMOD International Conference on Management of Data. pp. 145–156 (2002)